SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO1: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

CASE STUDY

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

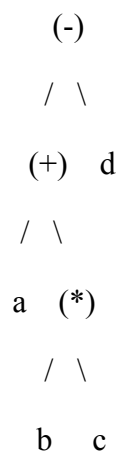
Signature of the student:

Q1. Syntax Tree and Syntax-Directed Definition for a + b * c - d

Given expression: a + b * c - d

The required precedence is * > + > -, and the operators are left associative. Therefore the expression is grouped as ((a + (b * c)) - d).

(i) Syntax tree



The root is '-' because the last operation performed is subtraction. Its left subtree represents a + (b*c), while its right child is d.

(ii) Syntax-Directed Definition (SDD)

E → E + T { E.node = Node('+', E1.node, T.node) }

E → E - T { E.node = Node('-', E1.node, T.node) }

E → T { E.node = T.node }

T → T * F { T.node = Node('*', T1.node, F.node) }

T → T / F { T.node = Node('/', T1.node, F.node) }

T → F { T.node = F.node }

F → (E) { F.node = E.node }

F → id { F.node = Leaf(id.lexeme) }

Here, node is a synthesized attribute. Node(op,left,right) creates an internal syntax-tree node, while Leaf(lexeme) creates a leaf node for an identifier.

(iii) Attribute evaluation

Step	Subexpression	Attribute / Result
1	b	Leaf(b)
2	c	Leaf(c)
3	b * c	Node(*, b, c)
4	a	Leaf(a)

5	a + (b*c)	Node(+, a, Node(*,b,c))
6	d	Leaf(d)
7	(a+b*c) - d	Node(-, Node(+,a,Node(*,b,c)), d)

Thus the final synthesized attribute E.node contains the complete syntax tree.

(iv) Representation of correct order of operations

- The '*' node is below the '+' node, so b*c is formed before addition.
- The '+' node is below the '-' root, so a+(b*c) is formed before subtraction.
- Because E → E + T and E → E - T are left recursive, + and - are left associative.
- The final structure therefore represents ((a + (b*c)) - d), which is the required evaluation order.

Q2. S-Attributed SDD, Bottom-Up Evaluation and Intermediate Code

Grammar:

S → E n

E → E + T | E - T | T

T → T * F | T / F | F

F → (E) | digit

Input: 8 + 2 * 4 n

(i) S-attributed SDD

S → E n { S.val = E.val }

E → E1 + T { E.val = E1.val + T.val }

E → E1 - T { E.val = E1.val - T.val }

E → T { E.val = T.val }

T → T1 * F { T.val = T1.val * F.val }

T → T1 / F { T.val = T1.val / F.val }

T → F { T.val = F.val }

F → (E) { F.val = E.val }

F → digit { F.val = digit.lexval }

All attributes are synthesized: the value of a parent is obtained from values of its children. Hence the definition is S-attributed and is suitable for bottom-up evaluation.

(ii) Translator/code fragment

```
int parseF() {
```

```
    if (lookahead is digit) return digit_value;
    if (lookahead == '(') {
        match('('); int v = parseE(); match(')'); return v;
    }
}

int parseT() {
    int v = parseF();
    while (lookahead == '*' || lookahead == '/') {
        char op = lookahead; match(op);
        int w = parseF();
        if (op == '*') v = v * w;
        else v = v / w;
    }
    return v;
}

int parseE() {
    int v = parseT();
    while (lookahead == '+' || lookahead == '-') {
        char op = lookahead; match(op);
        int w = parseT();
        if (op == '+') v = v + w;
        else v = v - w;
    }
    return v;
}
```

For intermediate three-address code, the expression can be translated as:

t1 = 2 * 4

t2 = 8 + t1

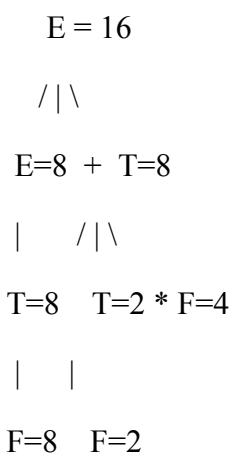
result = t2

(iii) Parser-stack demonstration

Step	Stack	Remaining Input	Action / Computation
1	$	8 + 2 * 4 n $	Shift 8
2	$ 8	+ 2 * 4 n $	Reduce digit → F; F.val=8
3	$ F	+ 2 * 4 n $	Reduce F → T; T.val=8
4	$ T	+ 2 * 4 n $	Reduce T → E; E.val=8
5	$ E	+ 2 * 4 n $	Shift +
6	$ E +	2 * 4 n $	Shift 2
7	$ E + 2	* 4 n $	Reduce digit → F; F.val=2
8	$ E + F	* 4 n $	Reduce F → T; T.val=2
9	$ E + T	* 4 n $	Shift *
10	$ E + T *	4 n $	Shift 4
11	$ E + T * 4	n $	Reduce digit → F; F.val=4
12	$ E + T * F	n $	Reduce T → T*F; T.val=2*4=8
13	$ E + T	n $	Reduce E → E+T; E.val=8+8=16
14	$ E	n $	Reduce S → E n; S.val=16

Final value = 16.

(iv) Syntax tree with computed values



The leaves are 8, 2 and 4. The multiplication subtree computes 2*4=8, and the addition then computes 8+8=16.

(v) Why bottom-up S-attributed evaluation preserves precedence and associativity

- The grammar places multiplication and division inside T, so they are reduced before E-level + or - operations.
- The E and T productions are left recursive, which gives left associativity.
- All values are synthesized from already reduced children, so no inherited value is required.
- For 8+2*4, the T subtree 2*4 is reduced first, producing 8, and only then is E→E+T reduced to obtain 16.

Q3. S-Attributed, L-Attributed and Dependency Analysis

Production: A → B C D. Each non-terminal has synthesized attribute s and inherited attribute i.

Definitions used: S-attributed definitions permit only synthesized attributes. L-attributed definitions permit inherited attributes subject to the left-to-right dependency restriction; a child's inherited attribute may depend on the parent's inherited attributes and attributes of symbols to its left, but not on symbols to its right.

(i) Consistency with S-attributed definition

Set	Rules	S-attributed?	Reason
Set (i)	A.s = B.i + C.i	No	B.i and C.i are inherited attributes.
Set (ii)	A.s = B.i + C.s; D.i = A.i + B.s	No	Inherited B.i and D.i occur.
Set (iii)	A.s = B.s + D.s	Yes	Only synthesized attributes are used.
Set (iv)	A.s = D.i; B.i = A.s + D.s; C.i = B.s; D.i = B.i + C.i	No	Several inherited attributes are used.

(ii) L-attributed analysis

Set	L-attributed?	Explanation
Set (i)	No	A.s depends on B.i and C.i. The inherited attributes are not defined by valid left-to-right rules in this set.
Set (ii)	No	D.i depends on B.s and A.i, which is structurally allowed for D, but B.i is not defined; therefore the complete set does not provide a valid evaluation.
Set (iii)	Yes	Only synthesized dependencies are present. It can be evaluated bottom-up and therefore also satisfies the L-attributed restriction.
Set (iv)	No	It contains a circular dependency: A.s → B.i → D.i → A.s.

(iii) Valid evaluation order / dependency analysis

Set (i): A.s requires B.i and C.i. Since these inherited attributes are not defined by the set, a complete evaluation order cannot be established from the given rules.

Set (ii): A.s can be computed from B.i and C.s, and D.i depends on A.i and B.s. However, B.i itself is not defined. Therefore the set is incomplete for evaluation.

Set (iii): B.s and D.s are available from their subtrees. Then A.s = B.s + D.s can be computed. There is no circular dependency.

Set (iv): The dependency cycle is A.s → B.i → D.i → A.s. Therefore no topological evaluation order exists.

(iv) Attribute dependency relationships

Set (iii): B.s ——————► A.s ◄—————— D.s

Set (iv):

A.s —► B.i —► D.i —► A.s

▲

|

C.i / B.i relationships

The key point is that a dependency graph must be acyclic for semantic attributes to be evaluated successfully.

(v) Circular dependency

Set (iv) contains a direct cycle: A.s depends on B.i, B.i depends on D.s and A.s, and D.i depends on B.i and C.i. In particular, A.s → B.i → D.i → A.s creates a circular dependency. Because none of the attributes in this cycle has a starting value independent of the cycle, the compiler cannot determine an evaluation order.

Q4. Type Propagation for the Declaration real p, q, r

Grammar:

D → T L

T → int | real

L → L , id | id

Input declaration: real p, q, r

The semantic analyzer must propagate the type obtained from T to every identifier in L.

(i) Suitable SDD

D → T L { L.in = T.type; }

T → int { T.type = 'int'; }

T → real { T.type = 'real'; }

L → L1 , id { L1.in = L.in;

id.type = L.in; }

L → id { id.type = L.in; }

Here L.in is an inherited attribute carrying the declared type from T to the identifier list. T.type is synthesized from the terminal int or real. The symbol table is updated whenever id.type is assigned.

(ii) Attributes and their roles

Attribute	**Type**	**Role**
T.type	Synthesized	Obtains the type from int or real.
L.in	Inherited	Carries T.type into the identifier list.
id.type	Assigned semantic information	Stores the declared type of each identifier.

For real p,q,r, T.type = real. This value is passed to L.in and then to p, q and r.

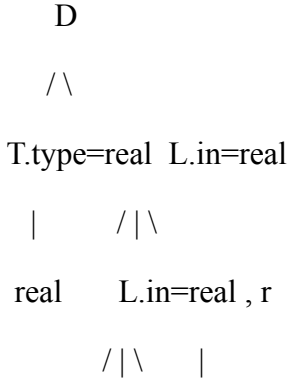
(iii) Bottom-up evaluation for real p, q, r

Reduction	**Attribute evaluation**	**Result**
T → real	T.type = real	T.type = real
L → id (p)	p.type = L.in	p.type = real
L → L , id (q)	L1.in = L.in; q.type = L.in	q.type = real
L → L , id (r)	L1.in = L.in; r.type = L.in	r.type = real
D → T L	L.in = T.type	All identifiers receive real

Final symbol-table entries:

Identifier	**Type**
p	real
q	real
r	real

(iv) Annotated parse tree



L.in=real , q r.type=real

/ \

p q.type=real

|

p.type=real

A clearer list structure is: L(p,q,r). The inherited value real flows from D/T to L and is copied to each id node: p.type=real, q.type=real, r.type=real.

(v) Stack implementation of bottom-up parsing

Step	Stack	Remaining input	Action / Attribute evaluation
1	$	real p , q , r $	Shift real
2	$ real	p , q , r $	Reduce T→real; T.type=real
3	$ T	p , q , r $	Shift p
4	$ T p	, q , r $	Reduce L→id; p.type=L.in=real
5	$ T L	, q , r $	Shift comma, q
6	$ T L , q	, r $	Reduce L→L,id; q.type=real
7	$ T L	, r $	Shift comma, r
8	$ T L , r	$	Reduce L→L,id; r.type=real
9	$ T L	$	Reduce D→T L; declaration completed

The type information is propagated semantically while the parser performs reductions. The important final result is that p, q and r are all entered in the symbol table with type real.

Q5. Top-Down Translation Using L-Attributed Definitions

Grammar:

S → E n

E → E + T | T

T → T * F | F

F → (E) | digit

Input: 3 * 4 + 6 n

For top-down translation, the left-recursive grammar is rewritten into an equivalent right-recursive form so that it can be handled naturally by a predictive/top-down parser:

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → (E) | digit

(i) L-attributed SDD

E → T E' { E'.inh = T.val; E.val = E'.syn; }

E' → + T E'1 { E'1.inh = E'.inh + T.val;

E'.syn = E'1.syn; }

E' → ε { E'.syn = E'.inh; }

T → F T' { T'.inh = F.val; T.val = T'.syn; }

T' → * F T'1 { T'1.inh = T'.inh * F.val;

T'.syn = T'1.syn; }

T' → ε { T'.syn = T'.inh; }

F → (E) { F.val = E.val; }

F → digit { F.val = digit.lexval; }

S → E n { S.val = E.val; }

The inherited attributes E'.inh and T'.inh carry partial results from left to right. The synthesized attributes E'.syn and T'.syn return the final values to their parent nodes. This satisfies the L-attributed left-to-right evaluation requirement.

(ii) Attribute propagation

Input: 3 * 4 + 6

F(3).val = 3

T'.inh = 3

read '*', F(4).val = 4

T'1.inh = 3 * 4 = 12

T'1 → ε, so T'.syn = 12

T.val = 12

E'.inh = 12

read '+', F(6).val = 6

E'1.inh = 12 + 6 = 18

E'1 → ε, so E'.syn = 18

E.val = 18

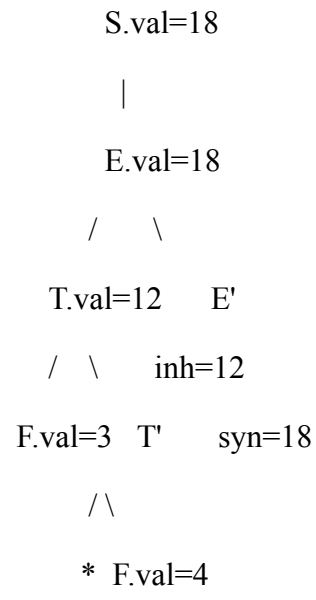
S.val = 18

(iii) Top-down translation and final value

Stage	Operation	Value
1	Read digit 3	F.val = 3
2	Process T	T'.inh = 3
3	Read * and digit 4	T'.1.inh = 3×4 = 12
4	Finish T	T.val = 12
5	Process E	E'.inh = 12
6	Read + and digit 6	E'.1.inh = 12+6 = 18
7	Finish E	E.val = 18
8	S → E n	S.val = 18

Final expression value = 18.

(iv) Annotated parse tree

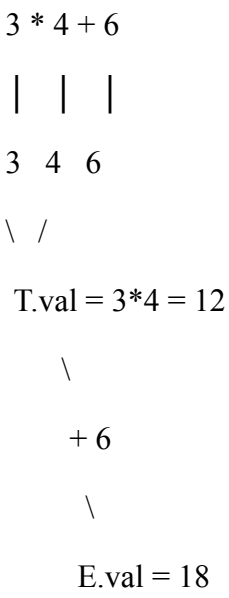


E' after +:

E'.inh=12 → + T(val=6)

→ E'.inh/syn = 18

A compact annotated representation of the evaluation is:



Operator precedence is preserved because multiplication is handled inside T before the + operation is processed by E. The expression is therefore evaluated as (3*4)+6 = 18.